Balancing FP8 Computation Accuracy and Efficiency on Digital CIM via Shift-Aware On-the-Fly Aligned-Mantissa Bitwidth Prediction

Liang Zhao, Kunming Shao, Zhipeng Liao, Xijie Huang, Tim Kwang-Ting Cheng, Chi-Ying Tsui, and Yi Zou

***Abstract*—FP8 low-precision formats have gained significant adoption in transformer inference and training. However, existing digital compute-in-memory (DCIM) architectures face challenges in supporting variable FP8 aligned-mantissa bitwidths, as unified alignment strategies and fixed-precision multiply accumulate (MAC) units struggle to handle input data with diverse distributions. This work presents a flexible FP8 DCIM accelerator with three innovations: 1) a dynamic shift-aware bitwidth prediction (DSBP) with on-the-fly input prediction that adaptively adjusts weight (2/4/6/8b) and input (2 ∼ 12b) aligned-mantissa precision; 2) a FIFO-based input alignment unit (FIAU) replacing complex barrel shifters with pointer-based control; and 3) a precision-scalable INT MAC array achieving flexible weight precision with minimal overhead. Implemented in 28-nm CMOS with a 64 × 96 CIM array, the design achieves 20.4 TFLOPS/W for fixed E5M7, demonstrating 2.8× higher FP8 efficiency than previous work while supporting all FP8 formats. Results on Llama-7b show that the DSBP achieves higher efficiency than fixed bitwidth mode at the same accuracy level on both BoolQ and Winogrande datasets, with configurable parameters enabling flexible accuracy–efficiency tradeoffs.**

***Index Terms*—Digital compute-in-memory (DCIM), mantissa extension, transformer, variable-mantissa FP8.**

I. Introduction

The continuous advancement of large language models (LLMs) has led to a significant increase in model parameters, making low-precision quantization crucial for accelerating computation and reducing power consumption. FP8, as a low-precision floating-point format, has seen widespread adoption for transformer inference and training [1], [2]. Compared with integer formats, FP8 leverages its exponent bits to provide a higher dynamic range than INT8 for outliers, enabling superior accuracy [3], [4]. However, under FP8 quantization, data with different distributions and formats (E2M5, E3M4, E4M3, and E5M2) coexist within the same network, requiring hardware support for variable aligned-mantissa bitwidths.

As shown in Fig. 1(a), we extracted the exponent values from three representative layers of FP8-quantized Llama-7b model under the optimal FP8 formats to analyze the data distribution. It can be observed that FP8 parameters in different formats have different exponent ranges and distributions. Furthermore, parameters of the same format extracted from different layers also exhibit differences in their distributions.

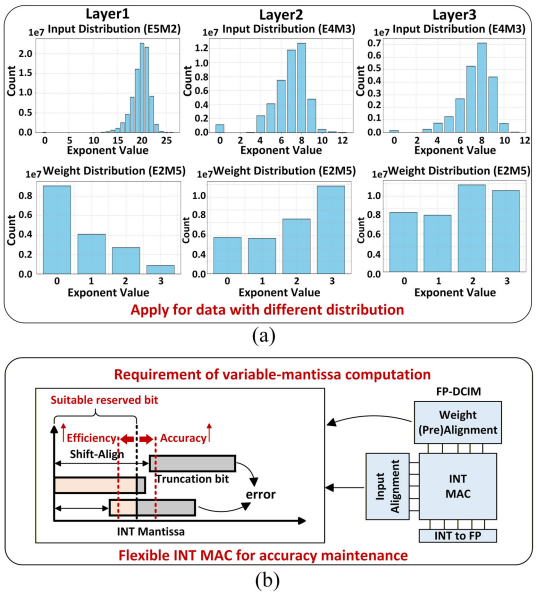


Fig. 1. (a) FP8 parameters extracted from Llama 7b with a different format. (b) Requirement of variable-mantissa computation based on FP-DCIM.

Compute-in-memory (CIM) architectures offer lower power consumption than traditional von Neumann architectures. As shown in Fig. 1(b), previous FP-CIM works [5], [6], [7], [8], [9] implement floating-point multiply-accumulate (MAC) using INT MAC units with exponent control, where mantissas are aligned to a fixed INT format by shifting according to exponent offset before computation. For example, [5] supports aligned-mantissa bitwidths of 8/16/24b for FP32/BF16/INT16/INT8 computations, while [6] aligns mantissas to 7/8b for INT8 and unsigned BF16 formats.

However, existing designs struggle to support FP8 formats with varying aligned-mantissa bitwidths. E4M3/E5M2 formats with larger exponent ranges require more aggressive shifts to represent activation outliers, making them susceptible to truncation errors. On the contrary, E2M5/E3M4 formats with smaller exponent ranges require narrower aligned bitwidths. This diversity creates a dilemma for fixed-bitwidth CIM designs: 1) large aligned-mantissa bitwidth [5], [8] minimizes truncation errors but degrades computational efficiency and 2) smaller aligned-mantissa bitwidth improves efficiency but cannot adapt to layer/channelwise varying FP8 formats [10], causing substantial accuracy loss. Moreover, continuous 1 ∼ 8b INT CIM arrays [11] incur significant precision-fusion overhead.

To address this, we propose a flexible CIM macro for FP8 inference with variable aligned-mantissa bitwidths, balancing accuracy and energy efficiency. First, dynamic shift-aware bitwidth prediction (DSBP) adaptively determines the aligned-mantissa bitwidth based on groupwise data distribution. Second, a mantissa prediction

Received 5 February 2026; revised 20 April 2026 and 12 May 2026; accepted 12 May 2026. Date of publication 2 June 2026; date of current version 27 July 2026. This work was supported in part by Shenzhen Science, Technology and Innovation Bureau Research Fund Research Fund under Grant KJZD20240903102708012; in part by the National Natural Science Foundation of China (NSFC) Fund under Grant U24B20151; and in part by the Artificial Intelligence Chip Center for Emerging Smart Systems (ACCESS) through InnoHK Funding, Hong Kong, SAR. *(Liang Zhao and Kunming Shao contributed equally to this work.) (Corresponding authors: Kunming Shao; Yi Zou.)*

Liang Zhao and Yi Zou are with South China University of Technology, Guangzhou 511442, China, and also with MoE Engineering Research Center of DTCO of Integrated Circuits, Guangzhou 511442, China (e-mail: 202321062080@mail.scut.edu.cn; zouyi@mail.scut.edu.cn).

Kunming Shao, Xijie Huang, Tim Kwang-Ting Cheng, and Chi-Ying Tsui are with The Hong Kong University of Science and Technology, Hong Kong SAR, China (e-mail: kshaoaa@connect.ust.hk; huangxijie1108@gmail.com; timcheng@ust.hk; eetsui@ust.hk).

Zhipeng Liao is with Westlake University, Hangzhou 310030, China (e-mail: liaozhipeng@westlake.edu.cn).

Digital Object Identifier 10.1109/TVLSI.2026.3693970

1063-8210 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:27 UTC from IEEE Xplore. Restrictions apply.

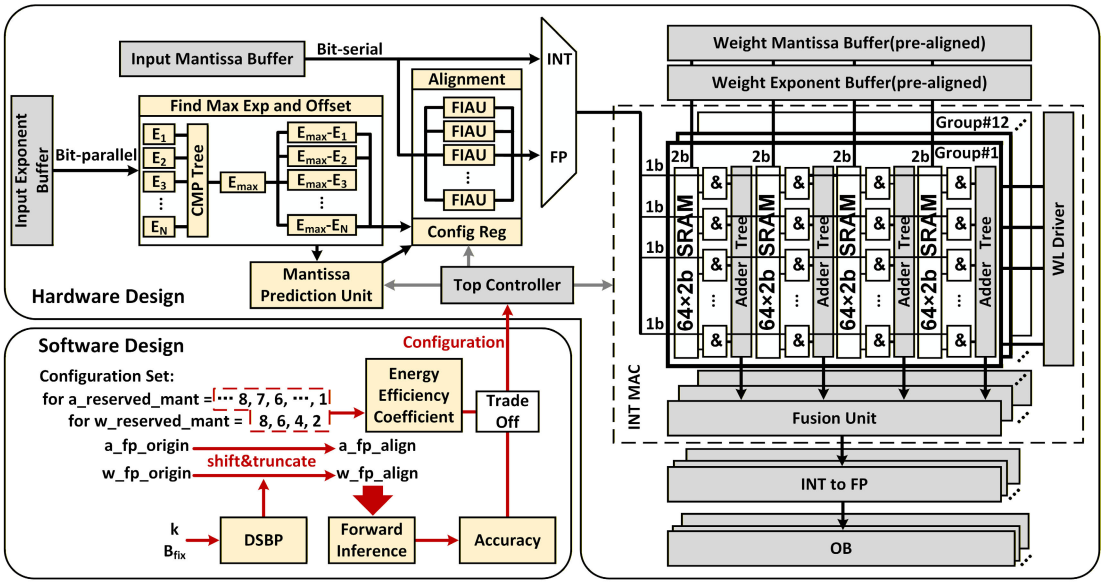


Fig. 2. Overall framework of our software–hardware codesign variable-mantissa FP8 DCIM accelerator.

unit (MPU) enables bitwidth prediction, while a FIFO-based input alignment unit (FIAU) executes 2 ∼ 12b input alignment. Third, a 2/4/6/8b INT MAC array supports flexible weight precision and efficient computation.

II. Proposed Design

Fig. 2 shows the overall hardware–software codesigned architecture. On the software side, the DSBP algorithm analyzes weight distributions offline to determine aligned-mantissa bitwidths and configures prediction hyperparameters for on-chip input bitwidth prediction. On the hardware side, the MPU predicts input-aligned bitwidths on-the-fly using the configured hyperparameters. The FIAU then performs mantissa alignment based on exponent offsets and the determined bitwidths. Finally, the aligned inputs and weights are processed by a 64 × 96 SRAM-based INT MAC array supporting 2/4/6/8b weight and 2 ∼ 12b input precision through flexible fusion paths.

A. Dynamic Shift-Aware Bitwidth Prediction

Different FP8 formats exhibit varying sensitivities to a certain fixed aligned-mantissa bitwidth and yield different truncation errors. To balance accuracy and efficiency, we propose DSBP, which dynamically determines the aligned-mantissa bitwidth. The aligned bitwidth is set to 1/3/5/7b for weights and 1 ∼ 11b for inputs, which is then extended by a 1-b sign bit to form the aligned-mantissa for INT MAC computation on CIM. **In DSBP, a group refers to the set of weights and inputs participating in the MAC operation in each column, determining the alignment granularity.** For a 64-row SRAM array, each group contains 64 elements.

The pseudocode of DSBP is shown in Algorithm 1. We embed this process into the MAC operation of the models. Based on the shift_i values of the elements, DSBP selects appropriate values for the aligned-mantissa bitwidths to balance efficiency and truncation error. For the dynamic predict result per group B_g, it consists of two parts, $B_{g,\text{dyn}}$ and B_{fix}. $B_{g,\text{dyn}}$ is dynamically determined by the shift_i values of the elements in each group. k and B_{fix} are hyperparameters that can be explored and adjusted offline under accuracy and efficiency requirements. B_g is then rounded to the valid integer value in the search space. For weights and activations, B_g is rounded to the nearest valid bitwidth of 1/3/5/7b and 1 ∼ 11b, respectively.

The shift_i value in $B_{g,\text{dyn}}$ reflects the contribution of each element to the MAC result. We use a weighted average based on the distribution of shift_i to obtain the final $B_{g,\text{dyn}}$. Since truncating element i incurs an error proportional to 2^{-shift_i}, setting $w_i = 2^{-\text{shift}_i}$ lets $B_{g,\text{dyn}}$ prioritize high-magnitude elements whose truncation causes the greatest accuracy loss. For example, if all exponent values are identical, all shift_i values are 0 in a group, and $B_{g,\text{dyn}}$ is 0, which means that the aligned-mantissa only needs B_{fix} bitwidth without further shifting. If almost all shift_i values are 3, $B_{g,\text{dyn}}$ will approach 3 to balance bitwidth and truncation error. If almost all shift_i values are 6, $B_{g,\text{dyn}}$ still tends to stay close to 3, as their diminished contribution to the accumulation result limits their influence on $B_{g,\text{dyn}}$. k fine tunes this prediction as a hyperparameter.

Algorithm 1 Dynamic Shift-Aware Bitwidth Prediction

Input parameter: Sign (S), Exponent (E), Exponent bitwidth (E_{bit}), Mantissa (M), Group Size (G), Scaling Factor ($k \in \{1, 1.5, 2, 2.5, 3\}$), Fixed Bitwidth ($B_{\text{fix}}$, activations: $\{1, 2, \ldots, 11\}$, weights: $\{1, 2, \ldots, 7\}$)
Output parameter: Aligned output tensor (**Y**), Predicted aligned-mantissa bitwidth tensor (**B**)
$M_{\text{bit}} = 7 - E_{\text{bit}}$
Partition S, E, M into groups of size G
for all group g **do**
  $E_{g,\max} = \max(E_g)$
  for all element i in group g **do**
    $\text{shift}_i = E_{g,\max} - E_{g,i}$
    $w_i = 2^{-\text{shift}_i}$
  end for
  $B_{g,\text{dyn}} = \frac{\sum_i \text{shift}_i \cdot w_i}{\sum_i w_i}$
  $B_g = \text{round_to_valid}(k \cdot B_{g,\text{dyn}} + B_{\text{fix}})$ {Weight: $\{1, 3, 5, 7\}$, Input: $\{1, \ldots, 11\}$}
  for all element i in group g **do**
    $M'_{g,i} = \text{round}(M_{g,i} \cdot 2^{B_g - M_{\text{bit}} - \text{shift}_i})$
    $Y_{g,i} = \text{sign}(S_{g,i}) \cdot M'_{g,i} \cdot 2^{E_{g,\max} - B_g}$
  end for
end for
return Y, B

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:27 UTC from IEEE Xplore. Restrictions apply.

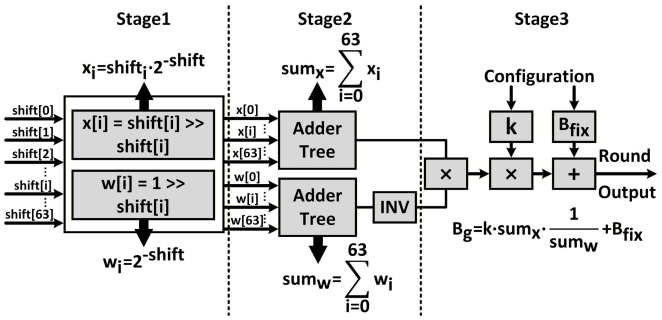


Fig. 3. Schematic of the proposed MPU.

B. MPU Design

The MPU implements the DSBP calculation described in Algorithm 1 for on-the-fly input-aligned-mantissa bitwidth prediction. Given 64 shift$_i$ values from one group, the circuit computes

$$B_g = k \cdot \frac{\sum_i \text{shift}_i \cdot 2^{-\text{shift}_i}}{\sum_i 2^{-\text{shift}_i}} + B_{\text{fix}}. \quad (1)$$

As shown in Fig. 3, the circuit employs a three-stage pipeline architecture. In Stage 1, 64-parallel shift units compute shift$_i \cdot 2^{-\text{shift}_i}$ and 2^{-shift_i} by shift$_i \gg$ shift$_i$ and $1 \gg$ shift$_i$, respectively. Stage 2 consists of two 64-input adder trees that aggregate the shifted results. In Stage 3, division is implemented using an 8-b reciprocal lookup table (LUT) to avoid expensive divider circuits. The final result is obtained by multiplying by k, adding B_{fix}, and saturating to 5b. The MPU occupies 7.0% of the total CIM macro area and is only activated in dynamic bitwidth mode. In fixed bitwidth configurations, the MPU is clock gated to eliminate power consumption, and B_g is directly set to the predetermined fixed value. The FIAU begins alignment concurrently with MPU computation. During B_g calculation, the FIAU shifts B_{fix} in parallel, and once B_g is returned, the FIAU determines the final bitwidth. This overlaps the MPU's three-stage computation with FIAU operations, avoiding additional latency overhead.

C. FIFO-Based Input Alignment Unit

Previous FP-CIM works [5], [9] use parallel barrel shifters for mantissa alignment, which incurs significant area overhead and complex combinational logic depth. To alleviate this issue, we propose an FIAU that implements alignment through pointer control.

As shown in Fig. 4, the mantissa is stored in 2's complement format and serially written from MSB to LSB, with r_ptr initially pointing to the MSB. When the read enable signal r_en is asserted, r_ptr remains at MSB for exp_offset + 1 cycles before advancing, implementing sign-extension equivalent to right-shift alignment. The save_len signal controls output precision: after save_len cycles, r_ptr jumps to w_ptr position for the next mantissa. This approach replaces complex barrel shifters with simple pointer logic, enabling flexible bitwidth support with minimal hardware overhead. Under the same input configuration, we measure that the 28 nm synthesized FIAU achieves 21.7% area reduction and 34.1% power reduction compared to barrel shifters while providing greater flexibility for variable bitwidth support.

D. Flexible Precision Scaling INT MAC Array

Precision scaling architectures [11] accumulate column results through shift-and-add operations to obtain the final result. We have also explored a flexible precision scaling INT MAC architecture in [12], featuring reusable output fusion units with continuous 2 ~ 8b precision scaling. However, the reconfigurability of precision fusion incurs significant area and power overhead. Based on these observations, this brief adopts a dedicated CIM-based INT MAC array for FP8 computation, supporting 2 ~ 12b input and 2/4/6/8b weight.

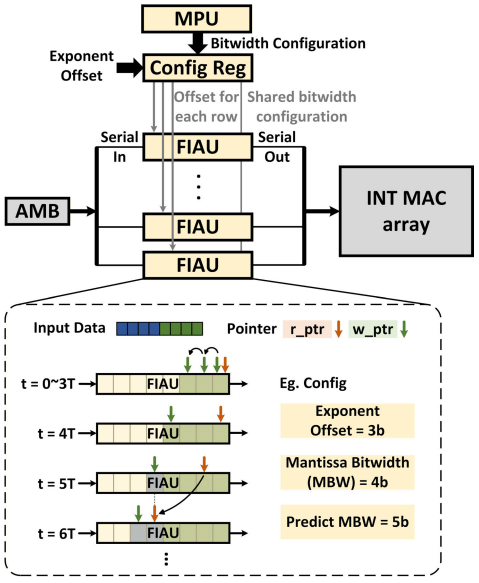


Fig. 4. FIAU achieves alignment by controlling pointer movement.

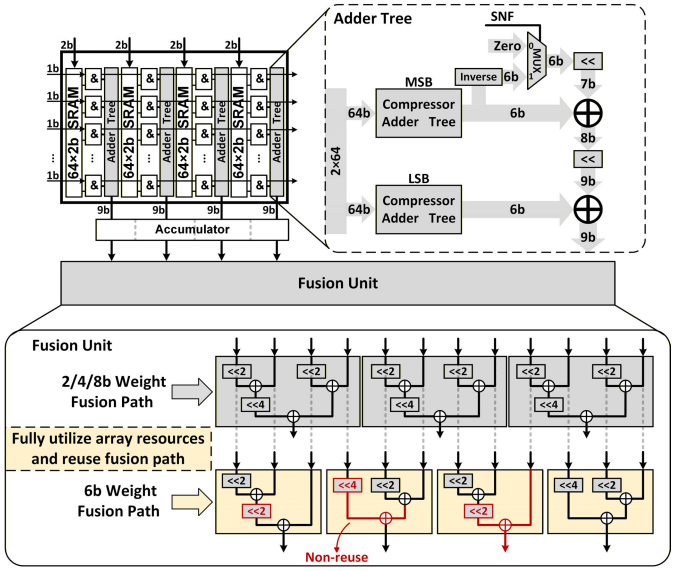


Fig. 5. Schematic of the adder tree and fusion unit.

As shown in Fig. 5, our fusion unit design maximizes circuit reuse across different bitwidth modes with 64 × 2b MAC for 2/4/6/8b weights. For 2/4/8b weight modes, scaling is achieved through the regular fusion path. However, the 6-b weight mode requires fusing three columns of 64 × 2b MAC results. To fully utilize the MAC array and reuse the fusion path, we introduce the 6-b weight fusion path. The red path indicates the additional circuit compared to the 2/4/8b weight fusion path, demonstrating that the 6-b weight path only brings a small amount of additional overhead. We use 4–2 compressors and full adders to form the adder tree for each MAC column [13]. The signed number flag (SNF) indicates whether the decomposed weights are signed, following the methods in [11] and [12].

III. Validations and Evaluations

The proposed FP-digital CIM (DCIM) is implemented in a 28 nm process: SRAM cells are designed in Cadence Virtuoso, logic modules are synthesized in synopsys design compiler, and placement and routing are performed in Cadence Innovus. Top-level integration and evaluation are completed in virtuoso. End-to-end software

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:27 UTC from IEEE Xplore. Restrictions apply.

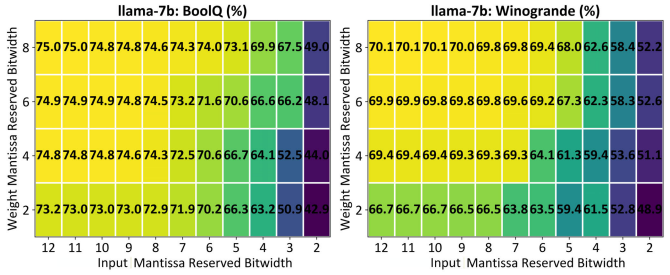


Fig. 6. Accuracy of Llama 7b on BoolQ and Winogrande. FP8 Baseline: 75.0% (BoolQ) and 70.1% (Winogrande). INT8 baseline: 64.3% (BoolQ) and 57.4% (Winogrande).

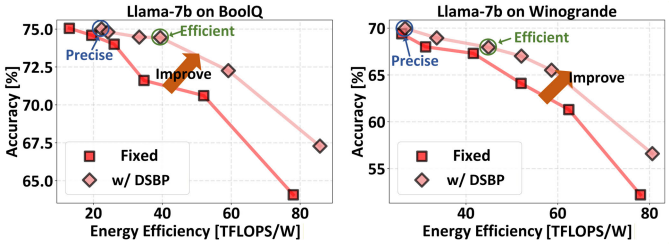


Fig. 7. Comparison of accuracy versus energy efficiency for fixed and DSBP on Llama 7b. (a) BoolQ dataset. (b) Winogrande dataset.

experiments are conducted on an NVIDIA RTX 4090 GPU, using an equivalent Python-based model for our macro.

A. Evaluations of Bitwidth and Accuracy

As shown in Fig. 6, we evaluate the accuracy of Llama 7b using a fixed aligned-mantissa bitwidth configuration as the baseline. Llama 7b is quantized following the method in [10], with inputs in E4M3/E5M2 format and weights in E2M5 format. Detailed accuracy versus bitwidth exploration on the BoolQ and Winogrande datasets demonstrates that 12-b input and 8-b weight achieve accuracy equivalent to the FP8 baseline, establishing the upper bound of our alignment scheme.

Fig. 7 illustrates the accuracy–efficiency tradeoff between fixed and dynamic mantissa strategies on Llama 7b. **We use I/W, including sign bits, to represent the average input/weight computational bitwidths in the INT MAC array.** The macroenergy efficiency is calculated as throughput divided by power, where throughput is inversely related to I $\times$ W. As shown in Fig. 7, through dynamic prediction, DSBP effectively balances groupwise bitwidth and truncation error, forming a Pareto frontier for accuracy and efficiency. This indicates that DSBP achieves higher energy efficiency at equivalent accuracy levels compared to the baseline. Furthermore, even though DSBP and MPU introduce additional area and power overhead, both energy efficiency and throughput improve compared to fixed bitwidth mode while maintaining accuracy.

Considering the diverse requirements of practical application scenarios, we define two configuration strategies: **Precise** and **Efficient**, as marked shown in Fig. 7. Table I presents detailed evaluation results on the BoolQ dataset. The scaling factor k and fixed bitwidth B_{fix} enable flexible tradeoffs between accuracy and efficiency. The **Precise** configuration ($k = 1$, $B_{\text{fix}} = 6/5$) prioritizes accuracy through conservative bitwidth reduction, while **Efficient** configuration ($k = 2$, $B_{\text{fix}} = 4/4$) aggressively reduces bitwidth for higher throughput. With a higher $B_{\text{fix}} = 6/5$, the precise configuration has a higher base bitwidth and yields a higher average bitwidth (BoolQ: 7.65/6.61, Winogrande: 7.02/6.02), maintaining accuracy close to the FP8 baseline. For the efficient configuration, increasing k to 2 and reducing $B_{\text{fix}} = 4/4$ amplifies the effect of shift-aware bitwidth prediction, achieving a lower average bitwidth (BoolQ: 5.58/6.08, Winogrande: 5.18/6.02) with higher efficiency compared to the precise configuration. This demonstrates that users can select appropriate (k, B_{fix}) hyperparameters based on their specific accuracy–efficiency requirements.

To validate the generalization capability of our method across different models and datasets, we also evaluate ResNet50 on ImageNet. The precise configuration achieves 76.1% accuracy, matching the FP8 baseline, while the efficient configuration achieves 74.7% accuracy with 1.5$\times$ higher energy efficiency compared to the precise configuration.

TABLE I
THROUGHPUT AND ENERGY EFFICIENCY COMPARISON

Format	Avg. I/W (bit)	k	B_{fix}	Throughput[3]	Efficiency[4]
E5M3[1]	4/4	0	3/3	0.192 TFLOPS	77.9 TFLOPS/W
E5M7[1]	8/8	0	7/7	0.048 TFLOPS	20.4 TFLOPS/W
INT4[1]	4/4	-	-	0.192 TOPS	109.3 TOPS/W
INT8[1]	8/8	-	-	0.048 TOPS	27.3 TOPS/W
Precise[2]	7.65/6.61	1	6/5	0.061 TFLOPS	22.5 TFLOPS/W
Efficient[2]	5.58/6.08	2	4/4	0.109 TFLOPS	39.4 TFLOPS/W

[1] 50% weight sparsity and 50% input toggle rate. [2] Llama-7b on BoolQ dataset. [3] Measured at 0.9V/250MHz. [4] Measured at 0.6V/50MHz.

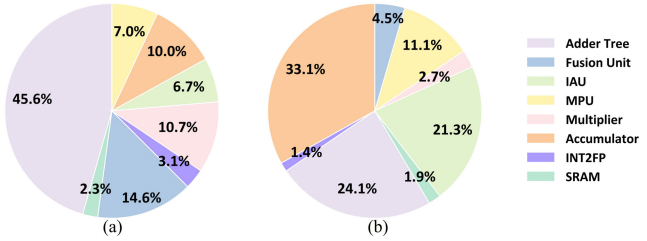


Fig. 8. (a) Area and (b) power breakdown of the proposed CIM macro.

B. Evaluations of the Proposed Hardware Architecture

The proposed FP-DCIM architecture features a 64 $\times$ 96 6T SRAM array, supporting variable aligned-mantissa bitwidths for weights and inputs, operating at 50 ~ 250 MHz and 0.6 ~ 0.9 V. Fig. 8 shows the area and power breakdown of our FP-DCIM macro. The input alignment unit includes FIAU, MPU, and maximum exponent processing logic. The fusion unit occupies 14.6% of the total area, with only 9.4% for the nonreused datapath, demonstrating minimal overhead. Table I presents postlayout throughput and energy efficiency. The fixed formats E5M3 and E5M7 are chosen because their I/W bitwidths (4/4b, 8/8b) correspond to common bitwidths in previous works [14], [15], enabling convenient comparison. Based on our macro, E5M3 achieves 4$\times$ higher efficiency than E5M7. INT8 reaches 27.3 TOPS/W, exceeding E5M7, as MPU, FIAU, and INT-to-FP circuits are disabled.

C. Comparison With Previous Works

Table II compares the proposed design with state-of-the-art FP-CIM macros. Diao et al. [6] and Saikia et al. [14] report postsilicon measurements, while [15] and this work present postlayout simulation results, with energy efficiencies scaled to 28 nm for fair comparison. This work achieves 20.4 TFLOPS/W for E5M7 (8/8b), 2.8$\times$ higher than [15] with 7.1 TFLOPS/W in E4M3 at the same 8/8b aligned bitwidth. Regarding FP8 format support, [14] only supports FP8 (E5M2) and BF8, which consists of an 8-b signed mantissa per value with a block-shared exponent, and [15] only supports E4M3 at 8/8b aligned bitwidth. On the contrary, our variable-aligned-mantissa architecture supports all FP8 formats from E2M5 to E5M2 at a

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:27 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON WITH SOTA FP-CIM ACCELERATORS

	CICC'24 [6](1)	**ESSCIRC'23** [14](1)	**ISCAS'25** [15](2)	**Ours**(2)
Process	28nm	28nm	40nm	28nm
Voltage	0.55-0.9V	0.55-1.2V	0.7-1.2V	0.6-0.9V
Freq.	20-180MHz	650MHz/2.4GHz	70-435MHz	50-250MHz
Area	0.143mm^2	0.71mm^2	1.876mm^2	0.052mm^2
SRAM	16Kb	4Kb	36Kb	6Kb
INT Prec.	8b	-	4/8b	I: 2-12b; W: 2/4/6/8
FP Prec.	UBF16	FP8(E5M2)/BF8	FP8(E4M3)	FP8(all format)
Peak Area Eff. (TOPS/mm^2 or TFLOPS/mm^2)	8.76(INT8) 7.02(UBF16)	0.94(FP8) 3.41(BF8)	0.46(INT8) 0.31(FP8)	0.923(INT8) 3.728(9)/0.923(10)(FP8)
Peak INT Eff.(3) (TOPS/W)	152(INT8)(4) @multiply-less NN	-	35.7(INT4)(5) 11.4(INT8)(5)	27.3(INT8)(5,8)
Peak FP Eff.(3) (TFLOPS/W)	128(UBF16)(4,10) @multiply-less NN	12.1(E5M2) 66.6(BF8)	7.1(E4M3)(5,10)	77.9(E5M3)(5,8,9) 20.4(E5M7)(5,8,10) 22.5(Precise)(11) 39.4(Efficient)(11)
Dynamic Mantissa	✗	✗	✗	✓
CNN Acc. (%)	ResNet50@ ImageNet: 76.2	-	-	ResNet50@ImageNet: 76.1(6)/ 74.7(7)
LLM Acc. (%)	DeiT-B@ ImageNet: 81.8	-	-	Llama-7b@BoolQ: 75.0(6)/ 74.5(7)

(1)Post-silicon results. (2)Post-layout results. (3)Scaled to 28nm and measured at the lowest voltage. (4)50% weight sparsity and 10% input sparsity. (5)50% weight sparsity and 50% input toggle rate. (6)DSBP precise configuration. (7)DSBP efficient configuration. (8)Our fixed configuration. (9)Aligned to 4/4b. (10)Aligned to 8/8b. (11)Measured on the Llama-7b with BoolQ dataset. ∗Since comparison with silicon-proven works is not fair, [6], [14] are included for reference purposes.

flexible aligned bitwidth. Furthermore, unlike previous works without dynamic bitwidth prediction, our design enables on-the-fly bitwidth adjustment based on input data distribution and offline adjustment for weights. Overall, the FIAU, compressor-based adder tree, and optimized fusion unit provide an efficient architecture for fixed FP8 computation, while DSBP with MPU further improves accuracy and efficiency.

IV. CONCLUSION

This brief presents a flexible FP8 DCIM accelerator supporting variable aligned-mantissa bitwidths for diverse FP8 formats. The proposed DSBP achieves an optimal balance between accuracy and efficiency. The precision-scalable INT MAC array supports 2/4/6/8b weight and 2 ∼ 12b input aligned-mantissa bitwidths with minimal overhead through reusable fusion paths. Implemented in 28-nm CMOS, the macro achieves 20.4 TFLOPS/W for fixed E5M7, delivering 2.8× higher FP8 efficiency than previous work. On-the-fly DSBP maintains accuracy while improving energy efficiency on Llama 7b and on ResNet50. These results demonstrate effective software–hardware codesign for efficient variable-mantissa FP8 computation in digital CIM architectures.

REFERENCES

[1] D. Guo et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.

[2] A. Yang et al., "Qwen3 technical report," 2025, *arXiv:2505.09388*.

[3] T. Blankevoort, A. Kuzmin, M. Nagel, J. Peters, Y. Ren, and M. V. Baalen, "FP8 quantization: The power of the exponent," in *Proc. Adv. Neural Inf. Process. Syst. 35*, 2022, pp. 14651–14662.

[4] P. Micikevicius et al., "FP8 formats for deep learning," 2022, *arXiv:2209.05433*.

[5] F. Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[6] H. Diao et al., "A 28 nm 128TFLOPS/W computing-in-memory engine supporting one-shot floating-point NN inference and on-device fine-tuning for edge AI," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2024, pp. 1–2.

[7] T.-H. Wen et al., "34.8 A 22 nm 16Mb floating-point ReRAM compute-in-memory macro with 31.2TFLOPS/W for AI edge devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 580–582.

[8] A. Guo et al., "A 28-nm 64-kb 31.6-TFLOPS/W digital-domain floating-point-computing-unit and double-bit 6T-SRAM computing-in-memory macro for floating-point CNNs," *IEEE J. Solid-State Circuits*, vol. 59, no. 9, pp. 3032–3044, Sep. 2024.

[9] S. Yan et al., "A 28-nm floating-point computing-in-memory processor using intensive-CIM sparse-digital architecture," *IEEE J. Solid-State Circuits*, vol. 59, no. 8, pp. 2630–2643, Aug. 2024.

[10] S.-Y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "LLM-FP4: 4-bit floating-point quantized transformers," 2023, *arXiv:2310.16836*.

[11] H. You, W. Li, D. Shang, Y. Zhou, and S. Qiao, "A 1–8b reconfigurable digital SRAM compute-in-memory macro for processing neural networks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1602–1614, Apr. 2024.

[12] L. Zhao, K. Shao, F. Tian, T. K.-T. Cheng, C.-Y. Tsui, and Y. Zou, "A flexible precision scaling deep neural network accelerator with efficient weight combination," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2025, pp. 1–5.

[13] K. Shao et al., "SynDCIM: A performance-aware digital computing-in-memory compiler with multi-spec-oriented subcircuit synthesis," in *Proc. Design, Autom. Test Eur. Conf. (DATE)*, Mar. 2025, pp. 1–7.

[14] J. Saikia, A. Sridharan, I. Yeo, S. Venkataramanaiah, D. Fan, and J.-S. Seo, "FP-IMC: A 28 nm all-digital configurable floating-point in-memory computing macro," in *Proc. IEEE 49th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2023, pp. 405–408.

[15] J. Bazzi, M. E. Fouda, and A. Eltawil, "Reconfigurable precision INT4–8/FP8 digital compute-in-memory macro for AI acceleration," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2025, pp. 1–5.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:38:27 UTC from IEEE Xplore. Restrictions apply.